Cloud Computing (CS3081) Mini Project: Sequence-Level Distillation of a Command Parser for Local Kubernetes

Puneet Kumar -- Roll No. 230101025

Repository: https://github.com/puneetkumar-2005/DistillationCommandParser Model release: https://github.com/puneetkumar-2005/DistillationCommandParser/releases/tag/student-v1

Introduction

This project distils a large teacher model, Phi-3.5-mini-instruct (3.8B), into a small student, EleutherAI/pythia-70m, for a command parser that returns exactly one JSON object with the keys intent, date, time and place. The student is served by a FastAPI service running on a local k3d Kubernetes cluster, with SeaweedFS as the object store. The work is organised in the seven tasks below, worth 100 marks in total.

A note on measurement provenance, stated up front because it affects how every number below should be read: all service, load and latency measurements in Tasks 3, 4 and 7 are CPU-only. The service image uses CPU-only torch, the pods are limited to 1 CPU and 1 GiB, and a k3d cluster cannot see the host GPU. The laptop GPU (RTX 3050) was used only for Task 1 teacher labelling and Task 2 training, because CPU labelling was measured at roughly 13-14 hours for 3,000 commands. Tasks 1 and 2 have no latency measurement in the marking, and the report says where the GPU was used wherever the result appears.

1. Storage and teacher labelling (12 marks)

The cluster is a single-node k3d cluster named cloud-mini. SeaweedFS runs in the data namespace with a 5 Gi persistent volume and an S3-compatible service, and two buckets, datasets and models, were created. The 3,000-command pool (commands_pool.jsonl) was uploaded to the datasets bucket.

The teacher is Phi-3.5-mini-instruct, run through Ollama at temperature 0 and seed 42, with a fixed prompt that states the four-key JSON format, the slot rules (copy exactly as written, shortest phrase, null if not mentioned, never invent) and three examples. Labelling was parallelised with two Kubernetes Jobs, one per shard: a command belongs to shard i when its 0-based line number in the pool satisfies line % 2 == i. Each worker wrote the teacher's reply unedited, as {id, text, teacher_raw}, to datasets/teacher/shard-i.jsonl. Both shards completed 1500/1500 records in 54 minutes.

Before cleaning, the shards were verified against the pool: missing 0, duplicates 0, extra 0. Cleaning then applied four rejection rules in order, one count per record, so the counts balance:

Outcome	Count
Input records	3000
(a) invalid JSON	448
(b) wrong keys	15
(c) bad intent	0
(d) bad slot	62
Duplicates	40

Outcome	Count
Kept	**2435**

3000 = 448 + 15 + 0 + 62 + 40 + 2435. The kept file, data/v1/train.jsonl, has sha256 210cdd1307b4c054c503db9189a821b91f6346aab7da2d54dd1b5e62738e617f, verified with sha256sum -c, and was uploaded to s3://datasets/v1/ with its checksum. Training used only the first 1,500 of the 2,435 kept records, as the assignment allows at most 1,500.

2. Training the student (20 marks)

The student is EleutherAI/pythia-70m (70M parameters). Training input is the exact teacher prompt plus the command, and the target is the teacher's JSON reply, with the loss computed only on the target tokens (prompt tokens masked). Hyperparameters: 3 epochs, batch size 8, learning rate 5e-5, maximum length 512, seed 42 for v1 and 43 for v2. Training is a plain PyTorch loop driven by training/config.yaml, and make train SUBSET=50 was verified to run on CPU in 86 seconds.

Run	Seed	Steps	Time	Epoch losses
Student v1	42	564	735 s	0.2911 / 0.1018 / 0.0417
Student v2	43	564	672 s	0.2726 / 0.0940 / 0.0382

Both models were trained on the laptop GPU; the training script is device-agnostic and the CPU subset run above proves it also runs on CPU. The v1 model (273 MB) is published as a GitHub Release asset, student-v1.zip (216.33 MiB, sha256 6c0452f4c276c1d61e61d9c41602abf634ba965fd6755385380383d1b6d2bf9a). Model weights are not in git and not in the service image; the v2 zip has sha256 da6c8d8cb4bdb0135ec9098896490b5267ac7a567a93b673664618f11e7e96ab. Both models are also stored in S3 under models/v1/ and models/v2/.

Evaluation used the 100-command validation set, scored with the kit's score.py. The table below is in the order validity, intent, date, time, place, full frame (percent):

Model	Validity	Intent	Date	Time	Place	Full frame
Teacher (Phi-3.5)	91	72	72	80	82	44
Student v1	100	77	77	84	86	39
Student v2	100	74	82	85	86	45
Untrained pythia-70m	95	16	5	3	81	0

The marking gate is JSON validity of at least 95% and full frame at least half of the teacher's (22). Both students pass it. Student v1 already beats the teacher on every slot and reaches 39/44 = 88.6% of the teacher's full frame; student v2 reaches 45, beating both v1 and the teacher. The untrained baseline is instructive: its validity is 95% and its place accuracy 81% even though it understands almost nothing (intent 16, date 5, time 3). Those two numbers are artefacts -- the gold place is usually null, and the extractor returns null -- while its full frame of 0 is the real signal. Validity alone does not prove understanding.

Distillation approach: sequence-level, not logit-based

We used sequence-level distillation, not logit-based distillation. Logit-based distillation trains the student to match the teacher's per-token probability distribution (soft targets). That needs the teacher's logits, but our teacher is Phi-3.5 served through Ollama, which returns only generated text and exposes no logits, so

matching soft targets was impossible with the teacher available. Instead the teacher generated one complete JSON reply per command at temperature 0 and seed 42, and the student was trained on those exact replies as hard targets, with loss only on the target tokens. This is simpler and works with a black-box teacher, but it has a real limitation: the student inherits the teacher's errors. For example, 448 of the 3,000 teacher replies were invalid JSON and had to be discarded in cleaning, and any wrong slot value the teacher produced became training data for the student. Sequence-level distillation can only be as good as the teacher's own outputs.

3. Container service (14 marks)

The service is a FastAPI application (service/app.py) listening on port 8000. POST /parse takes {"text": "..."} and returns exactly the four-key JSON, with headers X-Model-Version and X-Served-By (the pod name). GET /healthz returns 200 while the process is alive, and GET /readyz returns 503 until the model is loaded and 200 after. All configuration comes from environment variables only: MODEL_URI, MODEL_VERSION, S3_ENDPOINT, MAX_NEW_TOKENS, plus FAULT_INJECT. At startup the service downloads the model and tokenizer from object storage (all objects under MODEL_URI) into /tmp/model, with credentials taken from the environment only; in Kubernetes those come from a Secret.

Every model output is checked against schema.json (exact keys, one of the eight intents, slots null or strings). Invalid output returns HTTP 500 {"error": "invalid_model_output"}. Setting FAULT_INJECT=1 replaces the model output with "not json" before the check, and the service then returns 500 with that error while /readyz still returns 200 -- this was verified locally. The image is parser-service:v1, built from python:3.14-slim with CPU-only torch, no weights inside, using 1.85 GB of disk against the 2 GB limit.

Fifty sequential requests were run against the container at --cpus=1 --memory=1g. All 50 returned 200, taking 0.55-0.68 s each (p50 about 0.56 s, p95 about 0.62 s). Peak memory after the run was 434.2 MiB of 1 GiB, CPU 0.12%.

4. Kubernetes deployment, scaling and reliability (22 marks)

The app/ manifests deploy the parser into the app namespace: a Deployment with 2 replicas, image parser-service:v1 with imagePullPolicy IfNotPresent, resources requests 500m CPU / 512 Mi and limits 1 CPU / 1 GiB, a liveness probe on /healthz and a readiness probe on /readyz (initialDelaySeconds 30, period 10 s, readiness failureThreshold 18), a Secret holding the S3 credentials, a Service, an Ingress through Traefik mapping /parse to the service, and a HorizontalPodAutoscaler (min 1, max 2, target 70% CPU). The Service has no ready endpoints while the model is still loading, because /readyz is 503 until then. The deployment also sets lifecycle preStop sleep 15 s and terminationGracePeriodSeconds 45, added after the first rolling-update test (see below).

Load test (Locust, 4 users, via the Ingress, validation commands). The clean measurement is the second run, loadtest2: 324 requests, 0 failures, average 1462 ms, p50 1200 ms, p95 3000 ms, throughput 2.71 req/s, split across replicas 161/163 (49.7% / 50.3%). The first run is discarded as evidence: its p95 was 6200 ms, above the 5000 ms limit, but it was a warm-up run and a pod was replaced mid-test, so it does not represent steady state.

Metric	loadtest2
Requests	324
Failures	0
Average latency	1462 ms
p50 latency	1200 ms
p95 latency	3000 ms
Throughput	2.71 req/s
Per-replica split	161 / 163

Rolling update: changing MODEL_URI and MODEL_VERSION to v2 rolled the deployment out, and the rollout was rolled back to v1 afterwards. The first rolling-update test, which spanned both the update and the rollback inside one 5-minute window, recorded 748 requests with 10 x HTTP 502 (1.34% failures), above the 0.5% limit -- Traefik was routing to terminating pods. The fix was the preStop sleep 15 s plus terminationGracePeriodSeconds 45 added to the deployment. The re-run, rollout2 (update only, rollback after the test), recorded 698 requests, 0 failures, p50 1300 ms, p95 3400 ms, throughput 2.33 req/s, with versions v1 171 / v2 527. Recovery: a parser pod was deleted under load; its replacement was 1/1 Running within 60 s and the replica count returned to 2. That test recorded 378 requests, 0 failures, p50 1900 ms, p95 4900 ms (higher because one pod carried all traffic during the replacement).

Cold start (pod creation to Ready) was measured three times at 33 s, 33 s and 33 s, median 33 s. The figure is dominated by the readiness probe's initialDelaySeconds 30; the model download and load take only a few seconds. Each run measured one pod, because the HPA had scaled the Deployment to 1 replica during idle. An earlier cold-start measurement (6 s / 5 s / 5 s) was wrong -- the timer started after the delete, which itself blocks on the preStop -- and was discarded and re-measured.

Autoscaling: at idle the HPA reported CPU 0% of the 70% target with 1 replica, the idle pod at 4m CPU / 430 Mi. Under load it reported CPU 200% of target and scaled 1 to 2 replicas, the new pod Ready in 40 s; the busy pod hit 1000m CPU (its 1 CPU limit) and 616 Mi memory. After the load ended the HPA scaled back 2 to 1. A related finding, recorded in the Makefile notes: when make up re-applies the manifests with replicas 2, the HPA can scale the second pod back down within about 30 s, before it is Ready, because its scale-down recommendation has been stable for hours. make up therefore waits for the Deployment to be Available (one Ready pod) rather than for all pods to be Ready, and the final curl is the proof the stack is up.

5. Subnetting and network access control (10 marks)

From the block 10.20.0.0/16, subnets were allocated largest first, each placed immediately after the previous one, using the smallest prefix that fits (usable hosts = 2^(32 - prefix) - 2):

Subnet	Network	Prefix	Usable hosts
app	10.20.0.0	/22	1022
public	10.20.4.0	/24	254
data	10.20.5.0	/25	126
management	10.20.5.128	/27	30

No two subnets overlap. The NACL for the public subnet (where the load balancer lives) is in network/nacl.json. Rules are checked lowest number first and the first match decides. Inbound: rule 100 denies all traffic from the abusive range 198.51.100.0/24; 110 allows TCP 80 from anywhere; 120 allows

TCP 443 from anywhere; 130 allows TCP 22 only from the management subnet; 140 allows TCP 1024-65535 from the app subnet (replies). Outbound: 100 allows TCP 8000 to the app subnet; 110 denies TCP 8000 anywhere else; 120 allows TCP 1024-65535 anywhere (replies). A NACL is stateless, so each direction of a connection needs its own rule: the app's reply to a request on TCP 8000 arrives as a new inbound packet to an ephemeral port, and without inbound rule 140 it would hit the default deny. Rule order matters because the first match decides: the deny for the abusive range must sit below the general web allows, or a packet from 198.51.100.9 to TCP 443 would match the allow first and the block would be dead. The supplied simulator was run on the laptop against the sample packets: 6/6 correct (S01 allow, S02 deny, S03 allow, S04 deny for the abusive range, S05 allow, S06 deny). The hidden 20-packet scoring cannot be checked in advance.

6. Reproducibility and CI/CD (10 marks)

The repository has a Makefile with train, up and down targets. make up brings the full stack up from a fresh clone in three commands (git clone, cd, make up): it creates or starts the k3d cluster, builds and imports the service image, applies SeaweedFS, seeds the v1 model from the GitHub Release into S3, applies the app manifests, waits for the Deployment to be Available, and finishes with a curl to /parse. It completed in 2m23s on the laptop, ending with HTTP 200 and the correct JSON. make down deletes the cluster with k3d cluster delete.

make down: exit 0, cluster deleted in about 7 seconds. make up: exit 0 on the second run - the first run timed out at the 180-second SeaweedFS wait because the fresh node needed 2m25s to pull the SeaweedFS image; the re-run restored the full stack and the parser Deployment became Available. The curl run immediately after the deployment became Available returned 'no available server' (the Ingress had not yet picked up the new endpoints).

Eight unit tests cover the cleaner (normalisation, key/intent sets, one record per rejection class with counts that balance), the worker's environment parsing, and the shard verifier (a correct split passes, a missing id fails). They pass in under a second: 8 passed.

The GitHub Actions workflow (.github/workflows/ci.yml) runs, in order: lint (ruff) -> unit tests -> secret scan (gitleaks, nothing found) -> download the model from the GitHub Release -> install CPU torch and evaluate -> score.py --min-validity 95 as the quality gate. Two runs on main are green: 42458414578 in 2m3s and 42460797290 in 1m59s. The CI evaluation reproduced the local v1 scores exactly (validity 100, time 84, place 86, full frame 39, 100 examples, 0 missing predictions). A deliberately failed run is also linked: run 42460805060 (PR #1, --max-new-tokens 1 truncates the model output) failed at the "Evaluate and quality gate" step with all earlier steps green, and the PR was closed unmerged. AI_USE.md discloses the AI assistance used in this project.

7. Performance, cost and review (12 marks)

All figures in this section are CPU-only, from the runs above. Standard load test (loadtest2): p50 1200 ms, p95 3000 ms, throughput 2.71 req/s, error rate 0%. The p95 is inside the 5000 ms examiner limit with 2000 ms of margin. Per-pod, the busy pod reached 1000m CPU (its limit) and 616 Mi memory during the HPA test; the Task 3 container test peaked at 434.2 MiB of 1 GiB. Cold start median is 33 s, dominated by the readiness probe delay.

Cost uses the rate card in student_kit/data/rate_card.json. These are hypothetical on-demand prices for the exercise, not real provider prices: INR 3.50 per vCPU-hour and INR 0.45 per GiB-hour, and a replica is billed for its CPU and memory limits for every hour it runs, whether or not it is busy. Per replica that is Rs 3.50 x 1 CPU + Rs 0.45 x 1 GiB = Rs 3.95/hour. The load test ran 2 replicas, so Rs 7.90/hour. At 2.71 req/s the service handles 2.71 x 3600 = 9,756 requests per hour, so the cost is 7.90 / 9,756 x 1000 = Rs 0.81 per 1,000 requests. Cross-check with rollout2 (2.33 req/s = 8,388 requests/hour) gives 7.90 / 8,388 x 1000 = Rs 0.94 per 1,000 requests, the same order of magnitude, so the figure is stable. This assumes 2 replicas running; at 1 idle replica the same arithmetic gives Rs 0.40 per 1,000 at the same throughput -- the two figures must not be mixed.

Service level objective: 99% of /parse requests succeed, and p95 latency stays under 5000 ms. On the standard load test the service achieved 100% success (324 requests, 0 failures). The error budget at 99% is 1% x 324 = 3.24 requests; used 0. Latency p95 was 3000 ms against the 5000 ms limit, a margin of 2000 ms.

Review table (Task 7 marking: each row needs a measured number as evidence, a limitation found, and an improvement proposed):

Component	Evidence (measured)	Limitation found	Improvement proposed
Data and storage	2435 of 3000 kept; 448 invalid JSON discarded	Teacher errors are inherited; 448 replies unusable	Retry failed replies at labelling time; tighten the prompt
Model quality	v2 full-frame 45 vs teacher 44; validity 100%	Slot errors remain (date 82, time 85)	Train a third version with a different learning rate
Latency and throughput	p50 1200 ms, p95 3000 ms, 2.71 req/s, 0% errors	Pod saturates its 1 CPU limit under load	Raise the CPU limit or raise the HPA maximum above 2
Resource utilisation	Peak 434.2 MiB of 1 GiB; busy pod 1000m CPU / 616 Mi	CPU is the bottleneck; memory is half unused	Rebalance limits: more CPU, less memory per pod
Reliability and recovery	Rollout2 0% errors; recovery 0% errors; cold start 33 s median	Cold start is probe-delay dominated; single node, no multi-machine fault tolerance	Lower readiness initialDelaySeconds; multi-node cluster
Security and cost	Gitleaks clean; credentials only from a Secret; Rs 0.81 per 1000 requests	Prices are hypothetical; "local" placeholder credentials sit in the repo	Real secret management (e.g. sealed secrets); cost monitoring

Limitations

The cluster is a single node on one laptop, so there is no multi-machine fault tolerance: losing the node loses the service, and the recovery test only proves pod-level recovery. The first rolling-update test failed at 1.34% errors before the preStop fix, and the first cold-start measurement was wrong and had to be re-measured; both are disclosed above rather than hidden. The HPA holds 1 replica at idle even though the manifest says 2, because its minimum is 1; it scales to 2 under load. One transient HPA warning (FailedGetScale "Unauthorized", a metrics-server RBAC quirk) was observed while scaling still worked. The student was trained on teacher outputs, so it inherits the teacher's errors, and the hidden 300-command test set has not been run; the validation results above are the best available estimate of its quality.